47m left

ALL

1

2

2. FizzBuzz

Given a number *n*, for each integer *i* in the range from *1* to *n* inclusive, print one value per line as follows:

- If *i* is a multiple of both *3* and *5*, print *FizzBuzz*.
- If *i* is a multiple of *3* (but not *5*), print *Fizz*.
- If *i* is a multiple of *5* (but not *3*), print *Buzz*.
- If *i* is not a multiple of *3* or *5*, print the value of *i*.

Function Description

Complete the function *fizzBuzz* in the editor below.

fizzBuzz has the following parameter(s):

int n: upper limit of values to test (inclusive)

Returns: NONE

Prints:

The function must print the appropriate response for each value *i* in the set *{1, 2, ... n}* in ascending order, each on a separate line.

Constraints

- $0 < n < 2 \times 10^5$

▶ **Input Format for Custom Testing**

▼ **Sample Case 0**

Odd-Even-Transform Problem works on an array of integers (both positive, negative, and zero), by taking in a value (a whole number) that specifies the number of times the transformation has to be applied.

On an array of integers, the transformation that n number of times needs to occur:
- Adds three (+3) to each odd integer.
- Subtracts three (-3) from each even integer.

Example:
[3, 4, 9] means the array with integers 3, 4, 9 has to undergo transformation 3 times.
[3, 4, 9] -> [6, 1, 12] -> [3, 4, 9] -> [6, 1, 12]. So the result is [6, 1, 12].

ALL

1

2

2. Question 2

In the assembly line, the factory assembles three parts 'a' 'b' 'c' of a triangle toy. A valid toy is one where the two shorter sides added together are greater in length than the longest side.

There are two forms of valid triangles to identify.

- If 2 parts are of equal length, the form is 'Isosceles'
- If all 3 parts are of equal length, the form is 'Equilateral'

If a toy is not valid or is not one of those 2 forms, it is 'None of these'.

Example

triangleToy=['2 2 1', '3 3 3', '3 4 5', '1 1 3']

- The first triangle is valid and has 2 equal parts, so the form is 'Isosceles'
- The second is valid and has 3 equal parts, so the form is 'Equilateral'
- The third is valid but not one of the two specific forms.
- The fourth is not valid. The two sides of 1 unit length cannot meet if the third side is 3 units.

1. Question 1

ALL

Given *n* request ids as an array of strings, *requests*, and an integer *k*, after all requests are received, find the *k* most recent requests. Report the answer in order of most recent to least recent.

Example:

1

Suppose *n* = 5, *requests* = ["item1", "item2", "item3", "item1", "item3"], and *k* = 3.

2

Starting from the right and moving left, collecting distinct *requests*, there is "item3" followed by "item1". Skip the second instance of "item3" and find "item2". The answer is ["item3", "item1", "item2"].

Function Description

Complete the function *getLatestKRequests* in the editor below.

getLatestKRequests takes the following arguments:

str requests[n]: the request ids

int k: the number of requests to report

Implement a simple prototype service to find the difference between two JSON (**JavaScript Object Notation**) objects.

To keep the prototype simple, a JSON will contain only key-value pairs and no nested objects or arrays in it. Given two JSON strings, *json1* and *json2*, find the list of keys for which the values are different. If a key is present in only one of the JSONs, it should not be considered in the result. The list of keys should be sorted in lexicographically ascending order.

Example:

Suppose *json1* = "{"hello":"world","hi":"hello","you":"me"}" and *json2* = "{"hello":"world","hi":"helloo","you":"me"}"

The only common key where the values differ is "hi". Hence the answer is ["hi"].

Function Description

Complete the function *getJSONDiff* in the editor below.

getJSONDiff has the following parameter(s):

json1: the first JSON string
json2: the second JSON string

Returns

string[]: a sorted list of keys that have different associated values in the two JSONs

There is a shop with old-style cash registers. Rather than scanning items and pulling the price from a database, the price of each item is typed in manually. This method sometimes leads to errors. Given a list of items and their correct prices, compare the prices to those entered when each item was sold. Determine the number of errors in selling prices.

Example

products = ['eggs', 'milk', 'cheese']
productPrices = [2.89, 3.29, 5.79]
productSold = ['eggs', 'eggs', 'cheese', 'milk']
soldPrice = [2.89, 2.99, 5.97, 3.29].

	Price		
Product	Actual	Expected	Error
eggs	2.89	2.89	
eggs	2.99	2.89	1
cheese	5.97	5.79	1
milk	3.29	3.29	

The second sale of eggs has a wrong price, as does the sale of cheese. There are *2* errors in pricing.

Function Description

Complete the function *priceCheck* in the editor below.

2. Question 2

As part of the development of the "HackerCoin" cryptocurrency exchange platform, create a query that returns a list of Ethereum wallets and their balances based on transaction amounts.

Due to the dummy data nature of the development process, some wallets may end up with a negative balance. Be sure to exclude them from the report.

The result should have the following columns: *address* | *transactions* | *balance*.

- *address* - wallet address
- *transactions* - the number of *confirmed transactions* in a specific wallet; a confirmed transaction is one with *confirmations* ≥ *10*
- *balance* - the sum of all the *amount* for all confirmed transactions in a specific wallet

The result should be sorted in descending order by *balance*.

Note:

- Only transactions with ten or more confirmations should be included in the report

2. Question 2

For each element of an array, a *counter* is set to *0*. The element is compared to each element to its left. If the element to the left is greater, the absolute difference is subtracted from the counter. If the element to the left is less, the absolute difference is added to the counter. For each element of the array, determine the value of the counter. These values should be stored in an array and returned.

Example

n = 3, the number of elements

arr = [2, 4, 3]

- For *arr[0] = 2*, *counter* starts at *0* and there are no elements to the left so *counter = 0*.
- For *arr[1] = 4*, *counter* starts at *0* and then increases by *| 4 - 2 | = 2* at the first and only comparison: *counter = 2*.
- Testing *arr[2] = 3*, first against *4*, *counter = 0 - |3 - 4| = -1*, and then against *2*, *counter = -1 + |3 - 2| = 0*.
- The answer array is *[0, 2, 0]*.

1. Question 1

When multiple tasks are executed on a single-threaded CPU, the tasks are scheduled based on the principle of pre-emption. When a higher-priority task arrives in the execution queue, then the lower-priority task is pre-empted, i.e. its execution is paused until the higher-priority task is complete.

There are *n* functions to be executed on a single-threaded CPU, with each function having a unique ID between 0 and *n - 1*. Given an integer *n*, representing the number of functions to be executed, and an execution log as an array of strings, *logs*, of size *m*, determine the *exclusive times* of each of the functions. Exclusive time is the

ALL

1

2

Prisoners are standing on a Cartesian coordinate system present inside a jail. Each prisoner stands at a unique point with both the coordinates (x,y) being integers. Each prisoner should be at a position that forms a rectangle or square with 3 other prisoners.

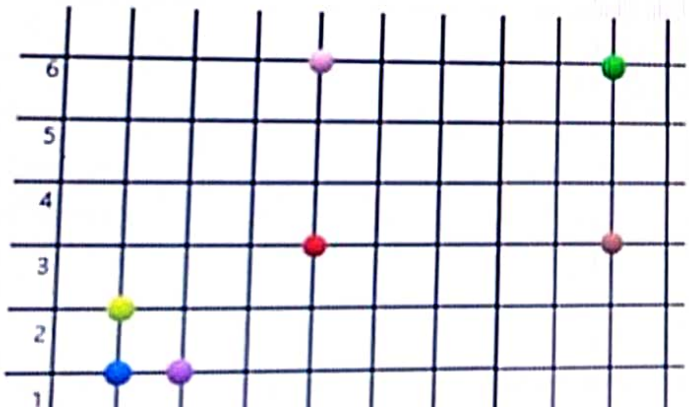
More formally, every prisoner at a coordinates pair *[x, y]* should have:

- at least one prisoner standing at integer coordinates *[x, ay]* where *ay != y*.
- at least one prisoner standing at integer coordinates *[bx, y]* where *bx != x*.

Unfortunately, one prisoner has escaped. Find the coordinates of that prisoner.

Example

locations = [[1, 1], [1, 2], [2, 1], [4, 4], [4, 6], [9, 4], [9, 6]]



1. Question 1

A password string, *pwd*, consists of binary characters (0s and 1s). A cyber security expert is trying to determine the minimum number of changes required to make the password secure. To do so, it must be divided into substrings of non-overlapping, even length substrings. Each substring can only contain 1s or 0s, not a mix. This helps to ensure that the password is strong and less vulnerable to hacking attacks.

Find the minimum number of characters that must be flipped in the password string, i.e. changed from 0 to 1 or 1

2. Question 2

For each element of an array, a *counter* is set to *0*. The element is compared to each element to its left. If the element to the left is greater, the absolute difference is subtracted from the counter. If the element to the left is less, the absolute difference is added to the counter. For each element of the array, determine the value of the counter. These values should be stored in an array and returned.

Example

n = 3, the number of elements

arr = [2, 4, 3]

- For *arr[0] = 2*, *counter* starts at *0* and there are no elements to the left so *counter = 0*.
- For *arr[1] = 4*, *counter* starts at *0* and then increases by *| 4 - 2 | = 2* at the first and only comparison: *counter = 2*.
- Testing *arr[2] = 3* first against *4*, *counter = 0 - |3 - 4| = -1*, and then against *2*, *counter = -1 + |3 - 2| =*

1. Question 1

The cost of a stock on each day is given in an array, *arr*. An investor wants to buy the stocks in triplets such that the sum of the cost for three days is divisible by *d*.
The goal is to find the number of distinct triplets *(i, j, k)* such that $i < j < k$ and the sum *(a[i]+a[j]+a[k])* is divisible by *d*.

Example

Let *arr, prices of stock = [3, 3, 4, 7, 8]* and $d = 5$. Following are the triplets whose sum is divisible by *d (1-based indexing)*:

- Triplet with indices - (1, 2, 3), sum = 3+3+4 which is equal to 10
- Triplet with indices - (1, 3, 5), sum = 3+4+8 which is equal to 15
- Triplet with indices - (2, 3, 4), sum = 3+4+8 which is equal to 15

Hence the answer is 3.

Given *n* request ids as an array of strings, *requests*, and an integer *k*, after all requests are received, find the *k* most recent requests. Report the answer in order of most recent to least recent.

Example:

Suppose *n* = 5, *requests* = ["item1", "item2", "item3", "item1", "item3"], and *k* = 3.

Starting from the right and moving left, collecting distinct *requests*, there is "item3" followed by "item1". Skip the second instance of "item3" and find "item2". The answer is ["item3", "item1", "item2"].

Function Description

Complete the function *getLatestKRequests* in the editor below.

getLatestKRequests takes the following arguments:

ALL

1

2

1. Question 1

Prisoners are standing on a Cartesian coordinate system present inside a jail. Each prisoner stands at a unique point with both the coordinates (x,y) being integers. Each prisoner should be at a position that forms a rectangle or square with 3 other prisoners.

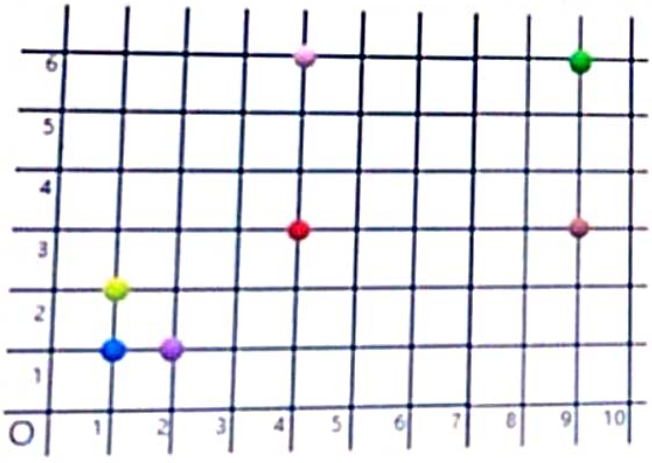
More formally, every prisoner at a coordinates pair *[x, y]* should have:

- at least one prisoner standing at integer coordinates *[x, ay]* where *ay != y*.
- at least one prisoner standing at integer coordinates *[bx, y]* where *bx != x*.

Unfortunately, one prisoner has escaped. Find the coordinates of that prisoner.

Example

locations = [[1, 1], [1, 2], [2, 1], [4, 4], [4, 6], [9, 4], [9, 6]]



ALL

1

2

1. Question 1

A Domain Name System (DNS) translates domain names to IP addresses which are then used by browsers to load internet resources. For quicker DNS lookups, browsers often store a number of recent DNS queries in a *DNS cache*. Retrieving data from the cache is often faster than retrieving it from a DNS server. This task aims to simulate DNS resolution and determine the time taken to process different URLs.

Assume that the DNS cache can store a maximum of the *cache_size* most recent DNS requests, i.e., URL-IP mappings. The cache is initially empty. It takes *cache_time* units of time to fetch data from the DNS cache, and *server_time* units of time to fetch data from the DNS server.

Given a list of *n* URLs visited as an array of strings, *urls*, determine the minimum time taken to resolve each DNS request.

Note: New DNS requests are dynamically added to

1. Tracks in Hackathon

There is a hackathon organized by HackerRank which has 2 separate tracks, healthcare and e-commerce. For track 1, the required team size is *teamSize_1*, while for track 2, the required team size is *teamSize_2*. The total number of participants is *p*.

Find the minimum number of teams into which the participants can be divided such that each participant belongs to exactly one team (either of track 1 or track 2), and each team has a size equal to either *teamSize_1* or *teamSize_2*. If no such division is possible, return -1.

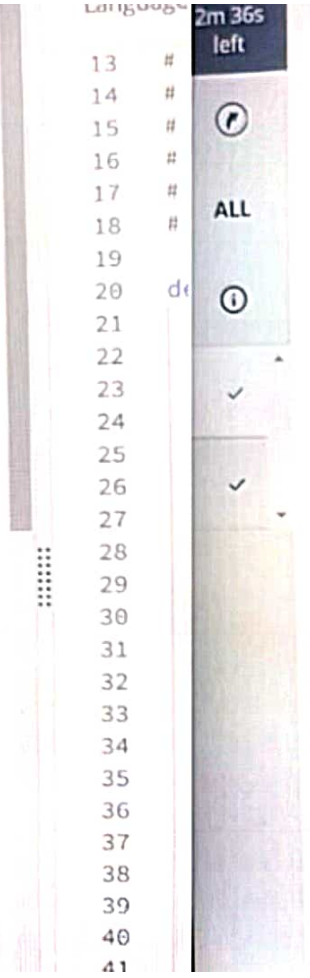
Example

Consider *teamSize_1 = 3*, *teamSize_2 = 4* and number of participants *p = 7*.

Optimally there is 1 team of 3 and 1 team of 4. The minimum number of teams is 2.

Function Description

Complete the function *countTeams* in the editor below.



Returns

int: the minimum number of teams into which the participants can be divided

Constraints

- $1 \le teamSize_1 \le 10^5$
- $1 \le teamSize_2 \le 10^5$
- $1 \le p \le 10^5$

▸ Input Format For Custom Testing

▾ Sample Case 0

Sample Input For Custom Testing

STDIN		FUNCTION
3	→	teamSize_1 = 3
4	→	teamSize_2 = 4
6	→	p = 6

Sample Output

2

Explanation

Optimally there are 2 teams of 3, and no teams of 4.

2. Array Reduction 3

Given an array *arr* of *n* integers, a sequence of *n-1* operations must be performed on the array.

In each operation,

- Remove the minimum and maximum elements from the current array and add their sum back to the array.
- The cost of an operation, *cost = ceil((minimum_element + maximum_element) / (maximum_element-minimum_element + 1))*.

Find the total cost to reduce the array to a single element.

Example

Given *arr = [2, 3, 4, 5, 7]*.

The possible sequence of operations are:

1. Choose 2 and 7, the *cost = ceil((2 + 7) / (7 - 2 + 1))* = *ceil[9 / 6] = 2*. Remove 2 and 7, append 9, *arr = [3, 4, 5, 9], total_cost = 2*.
2. Choose 3 and 9, the *cost = ceil((3 + 9) / (9 - 3 + 1))* = *ceil[12 / 7] = 2*, *arr* = [4, 5, 12], *total_cost* = 2 + 2 = 4.
3. Choose 4 and 12, the *cost* = *ceil((4 + 12) / (12 - 4 + 1))* = *ceil[16 / 9] = 2*, *arr* = [5, 17], *total_cost* = 4 + 2 = 6.
4. Choose 5 and 17, the *cost* = *ceil((5 + 17) / (17 - 5 + 1))* = *ceil[22 / 13] = 2*, *arr* = [22], *total_cost* = 6 + 2 = 8.

4m left

ALL

2

▸ Input Format For Custom Testing

▾ Sample Case 0

Sample Input For Custom Testing

```
STDIN       FUNCTION
-----       --------
6       →   arr[] size n = 6
3       →   arr = [3, 5, 2, 1, 9, 6]
5
2
1
9
6
```

Sample Output

```
10
```

Explanation

The possible sequence of operations are:

1. Choose 1 and 9, the *cost = ceil((1 + 9) / (9 - 1 + 1))* = *ceil[10/ 9] = 2*. Remove 1 and 9, append 10, *arr = [3, 5, 2, 6, 10], total_cost = 2*.
2. Choose 2 and 10, the *cost = ceil((2 + 10) / (10 - 2 + 1))* = *ceil[12 / 9] = 2*, *arr* = [3, 5, 6, 12], *total_cost* = 2 + 2 = 4.
3. Choose 3 and 12, the *cost = ceil((3 + 12) / (12 - 3 + 1))* = *ceil[15 / 10] = 2*, *arr* = [5, 6, 15], *total_cost* = 4 + 2 = 6.
4. Choose 5 and 15, the *cost = ceil((5 + 15) / (15 - 5 + 1))* = *ceil[20 / 11] = 2*, *arr* = [6, 20], *total_cost* = 6 + 2 = 8.
5. Choose 6 and 20, the *cost = ceil((6 + 20) / (20 - 6 + 1))* = *ceil[26 / 15] = 2*, *arr* = [26], *total_cost* = 8 + 2 = 10.

ALL

2

▾ Sample Case 1

Sample Input For Custom Testing

```
STDIN       FUNCTION
-----       --------
4       →   arr[] size n = 4
8       →   arr = [8, 8, 8, 8]
8
8
8
```

Sample Output

```
21
```

Explanation

The possible sequence of operations are:

1. Choose 8 and 8, *cost = ceil((8 + 8) / (8 - 8 + 1))* = *ceil[16 / 1] = 16*, *arr* = [8, 8, 16], *total_cost* = 16.
2. Choose 8 and 16, *cost = ceil((8 + 16) / (16 - 8 + 1))* = *ceil[24 / 9] = 3*, *arr* = [8, 24], *total_cost* = 16 + 3 = 19.

ALL

Two strings are given, *word* and *substr*. Some of the characters in *word* are a question mark (?). Find the lexicographically smallest string that can be obtained by replacing '?' characters such that *substr* appears at least once. If it is not possible to do so, return "-1".

1

Note:

2

- A substring is a contiguous sequence of characters within a string. For example, "bcd" is a substring of "abcde" but "ac" is not.
- For two strings a and b of the same length, a is lexicographically smaller than b if $a_i < b_i$ for some $0 \leq i < |a|$, and $a_j = b_j$ for all $0 \leq j < i$.

Example

word = "as?b?e?gf"

substr = "dbk".

*Replace the 3rd and 5th characters with 'd' and 'k' to get "as**dbk**e?gf"* which has *substr = "dbk"* as a substring. Replace the remaining '?' with 'a'. The final string is *"asdbkeagf"*.